

Memory Management API

CS232 Operating Systems

Fall 2019

Taj Muhammad Khan

Memory Management in Unix/C Programs

- Stack
 - Automatic management (By compiler)
 - Stores function params, local variables, return address, etc.
 - Memory gets freed once function call ends

```
void func() {  
    int x; // declares an integer on the stack  
    ...  
}
```

Memory Management in Unix/C Programs

- Heap

- Explicitly handled by the programmer

```
int *x = malloc(10 * sizeof(int));  
...  
free(x);
```

- Use sizeof to calculate size
 - But beware:

```
int x[10];  
printf("%d\n", sizeof(x));
```

```
int *x = malloc(10 * sizeof(int));  
printf("%d\n", sizeof(x));
```

Common Errors in Memory Management

- Failure to allocate memory

```
char *src = "hello";  
char *dst;           //  
strcpy(dst, src); //
```

- Not allocate enough memory

```
char *src = "hello";  
char *dst = (char *) malloc(strlen(src));  
strcpy(dst, src); // work properly
```

- Forgetting to initialize memory
- Forgetting to free memory
- Using memory after freeing (dangling pointer)
- Freeing memory repeatedly (double free)
- Calling free with invalid (not malloc'ed) address

System calls

- Malloc() and free() both are library calls
- Other library calls include calloc(), realloc(), etc.
- malloc() and free(), etc., use the brk() system call
 - brk, sbrk change the size of the heap
- mmap() is another system call for requesting memory from OS
 - mmap creates an anonymous memory region in your program